Code coverage report

1. Overview

The JaCoCo coverage report for the cardio_generator project shows an overall instruction coverage of 48% (1,271 of 2,453 instructions missed) and a branch coverage of 61% (77 of 198 branches missed). The report covers 22 classes and 92 methods in total. Coverage is strong in the data management and alerts packages but entirely absent in the simulation and output packages, as explained in the sections below.

2. Well-tested packages

2.1 com.alerts

This package achieved the highest coverage in the project, with 95% of instructions and 80% of branches covered. It contains 3 classes and 19 methods, all of which were reached by the test suite (0 missed methods, 0 missed classes). This package was given the highest testing priority as it contains the core alert evaluation and triggering functionality required by the assignment.

2.2 com.data_management

This package reached 84% instruction coverage and 80% branch coverage across its 7 classes and 26 methods. Only 2 methods and 0 classes were missed entirely. The untested instructions correspond primarily to exception-handling paths that are difficult to trigger without injecting errors artificially.

3. Untested packages

3.1 com.cardio_generator.generators

This package contains all patient data generator classes. The report records 504 missed instructions, 16 missed branches, 104 missed lines, and all 16 methods across all 5 classes entirely uncovered.

It was not tested for the following reasons:

- The generators rely heavily on randomised output via Math.random() and java.util.Random. Without seeding or mocking the random source, any assertion on their output would be non-deterministic and produce flaky tests.
- Most generator methods schedule recurring tasks using ScheduledExecutorService, making them time-dependent and impractical to control within a standard unit test lifecycle.

3.2 com.cardio_generator

This package contains the HealthDataSimulator class, the main simulation orchestrator. The report shows 340 missed instructions, 24 missed branches, 83 missed lines, and all 13 methods across its single class uncovered.

It was not tested for the following reasons:

- The class entry point (main()) starts multiple scheduled thread pools and runs an indefinite simulation loop.
- Its sole responsibility is wiring together the generators and output strategies; testing it in isolation provides limited value compared to testing the components it orchestrates.

3.3 com.cardio_generator.outputs

This package implements the output strategy classes. The report records 293 missed instructions, 4 missed branches, 62 missed lines, and all 16 methods across all 5 classes uncovered.

It was not tested for the following reason:

- Each class depends directly on external I/O resources: a writable file system directory, an open TCP port, or a running WebSocket server. Reliable, isolated unit tests would require mocking or stubbing these resources, or setting up a dedicated test server.

3.4 com (root package)

The root package contains a single entry-point wrapper class with 27 missed instructions and 4 missed branches. It contains no independent project logic, making it unsuitable for unit testing.

4. Conclusion

The parts of the codebase directly related to the assignment deliverables — alert generation, alert evaluation, data storage, and the DataReader implementation — are covered at 84–95% instruction coverage. The untested packages (generators, outputs, and the main simulator) were excluded due to their inherent dependency on randomness, external I/O, … all of which make standard unit testing impractical without significant additional infrastructure. The full breakdown of coverage figures can be found in the accompanying JaCoCo HTML report (index.html).